计算机领域本科教育教学改革试点
工作计划（“101计划”）研究成果

数据结构

授课教师：谭光华

湖南大学　信息科学与工程学院

第 9 章

不相交集

提纲

9.1 问题引入

9.2 等价关系、等价类和不相交集

9.3 不相交集的存储实现

9.4 不相交集的基本运算实现

9.5 不相交集的应用

9.6 拓展延伸

9.7 应用场景

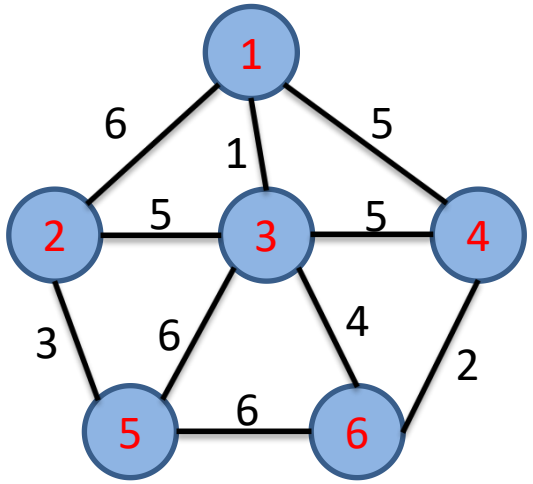
9.1 问题引入：Kruskal算法的高效实现

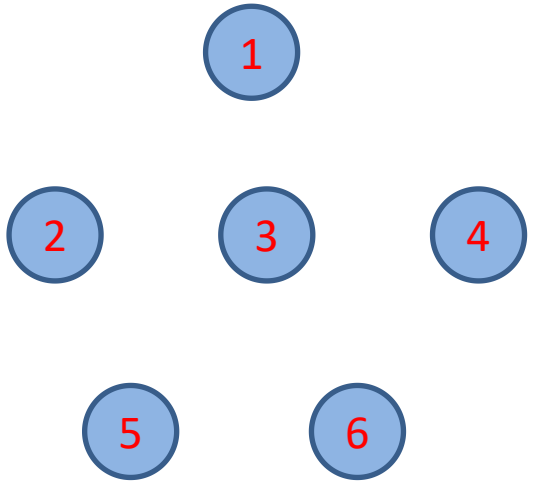
- **Kruskal算法**
  - 首先，将顶点集分为|V| 个等价类，每个等价类包括一个顶点
  - 然后，按照权重大小从小到大的顺序处理各条边。
    - 如果某条边连接两个不同等价类的顶点，则该边的加入不会引入回路，把这条边加入MST ，合并两个等价类；
    - 反复执行此过程直至只余下一个等价类。

每次查找权值最小的边，若不会引入回路，则加入最小生成树

Kruskal算法图示

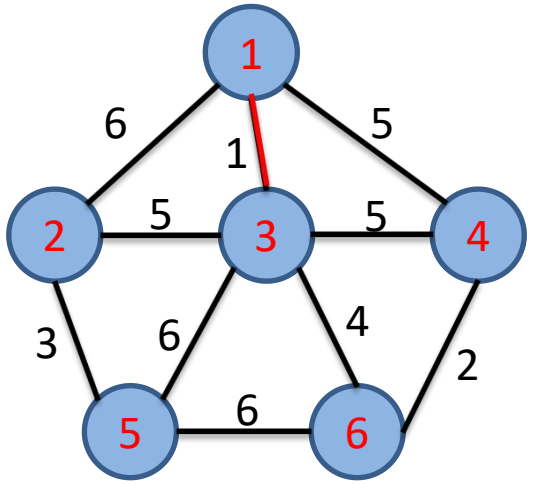
- **图示**
  - **初始连通分量：{1}，{2}，{3}，{4}，{5}，{6}**

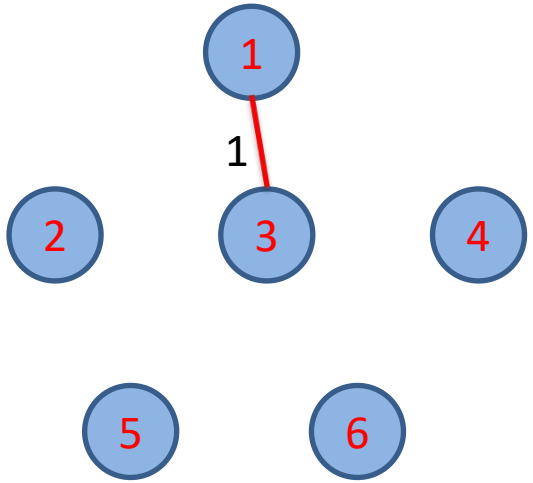




Kruskal算法图示

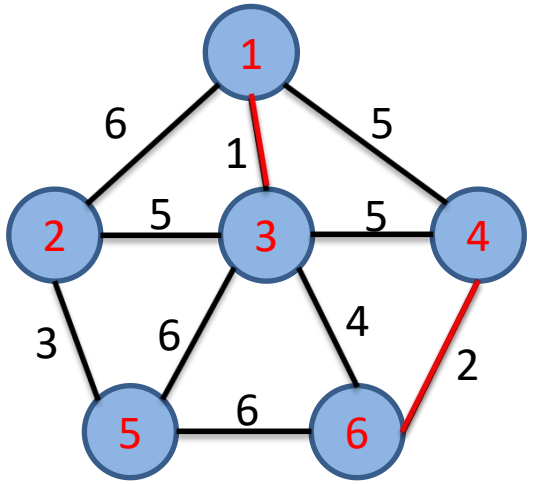
- **图示**
  - **选取边(1，3)：{1，3}，{2}，{4}，{5}，{6}**

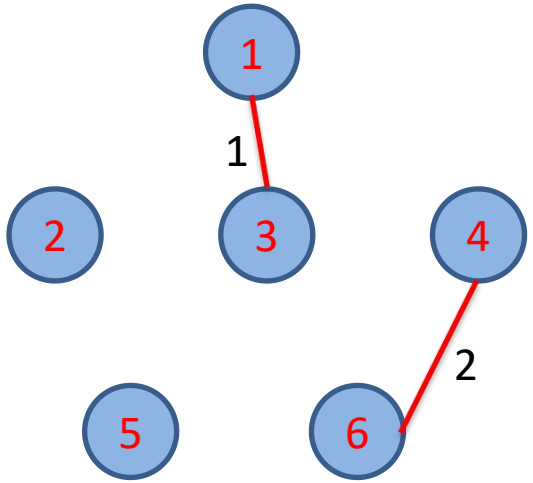




Kruskal算法图示

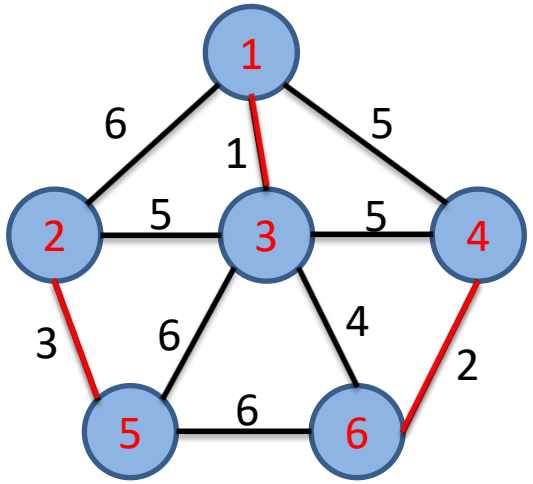
- **图示**
  - **选取边(4，6)：{1，3}，{2}，{4，6}，{5}**

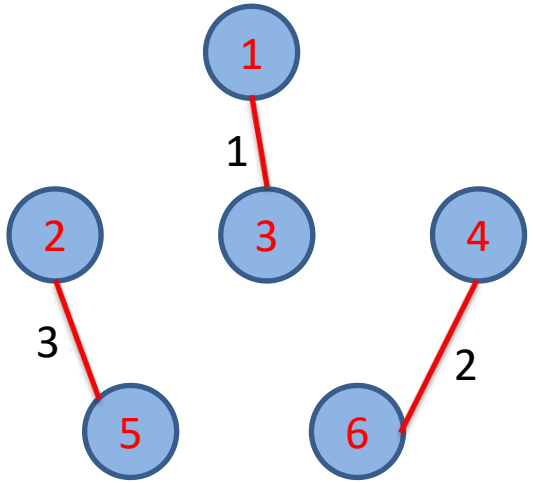




Kruskal算法图示

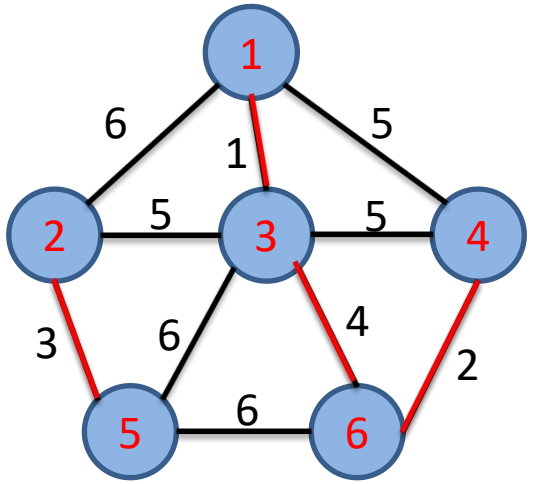
- **图示**
  - **选取边(2，5)：{1，3}，{2，5}，{4，6}**

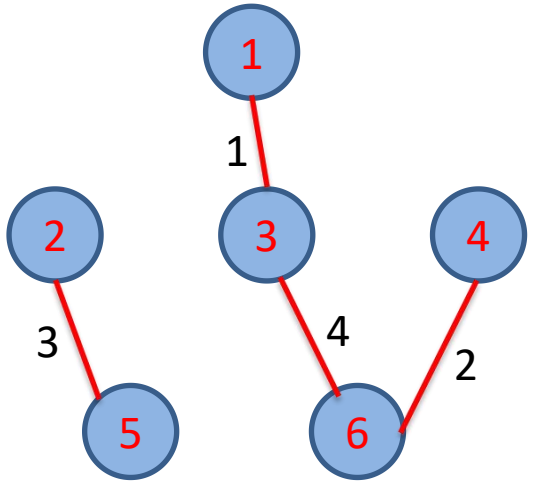




Kruskal算法图示

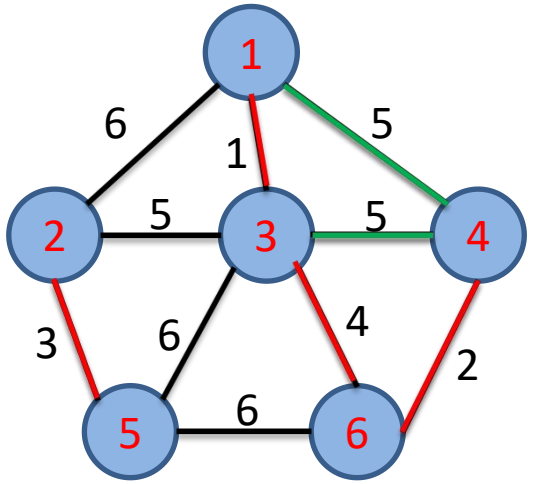
- **图示**
  - **选取边(3，6)：{1，3，4，6}，{2，5}**

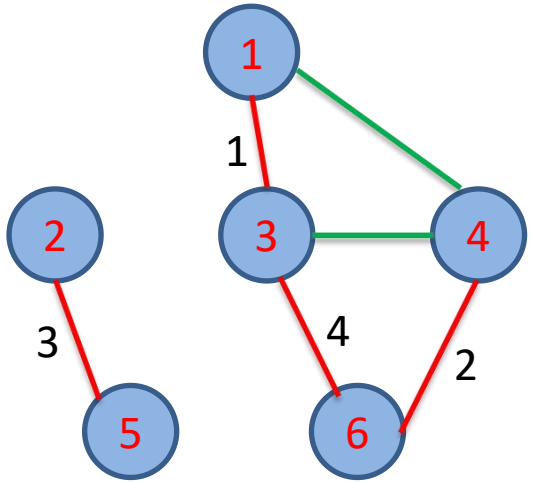




Kruskal算法图示

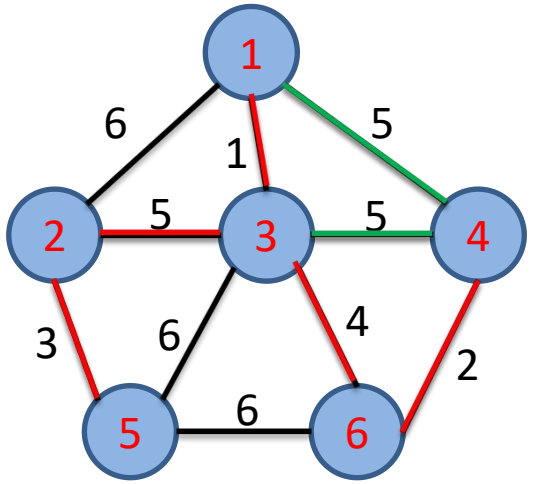
- **图示**
  - **选取边(1，4)：会构成回路**
  - **选取边(3，4)：会构成回路**

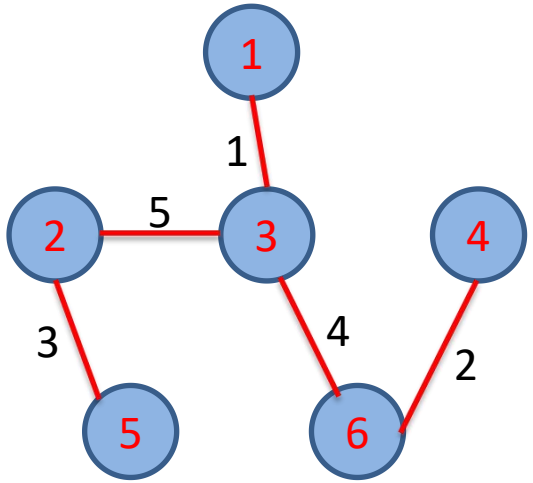




Kruskal算法图示

- **图示**
  - **选取边(2，3)：{1，2，3，4，5，6}**





9.1 问题引入：Kruskal算法高效实现

- **判断是否存在回路的直接做法**
  - **可通过图的遍历实现**
    - **每次判断需要O(|V|)**
  - **最坏情况下，加入每条边均需进行一次回路存在与否的判断**
    - **总的时间复杂度为O(|V||E|)**

9.1 问题引入：Kruskal算法高效实现

- **高效实现**
  - **只维护每个连通分量的顶点集合**

初始时，每个顶点都是独立的连通分量，此时有n个仅包含单一顶点的集合：$\{v_1\},\{v_2\},\ldots,\{v_n\}$

每加入一条边(u,v)，就将u所属的集合和v所属的集合合并

对于**是否存在回路的判断，只需查询u和v是否在同一集合**中

- **是否存在回路的判断转换成了维护若干不相交的集合，并动态地合并（Union）、查找（Find）的问题**

9.1 问题引入：Kruskal算法高效实现

- **利用不相交集的数据结构，合并、查找的操作均只需 $O(\alpha(n))$ 的时间复杂度**
  - **其中 $\alpha(\cdot)$ 是一个增长极其缓慢的函数，接近于常数函数**
  - **该时间复杂度仅略高于 $O(1)$，而大大低于图的遍历所需的 $O(n)$。**

9.2 等价关系、等价类和不相交集

- 不相交集与数学中等价关系、等价类的概念密切相关
  - 元素之间的等价关系自然地定义了若干不相交集的集合
  - 不相交集常常用于处理等价性查询的问题
    - 两个顶点在同一连通分量中就可以看做一种等价性

9.3 不相交集的存储实现

- **不相交集数据结构定义：**
  - **给定n个元素组成的集合X。不失一般性，令$X = \{1,2,\ldots,n\}$。不相交集的数据结构动态维护了集合X的一组划分，即若干个不相交的集合$X_1,\ldots,X_m$，满足:**
    - $\forall i \neq j, X_i \cap X_j = \emptyset$;
    - $\cup_{i=1}^{m} Xi = X$。
  - **例：考虑集合X={1, 2, 3, 4, 5, 6, 7, 8}。**
    - **合法的划分：{{1, 2, 3, 4, 5}, {6, 7}, {8}}或{{1, 2}, {3}, {4}, {5}, {6}, {7}, {8}}；**
    - **不合法的划分：{{1, 2, 3, 4, 5}, {6, 7}}和{{1, 2, 3}, {3, 4, 5}, {5, 6, 7, 8}}**

9.3 不相交集的存储实现

代码 9-1 不相交集的抽象数据类型定义

ADT DisjointSet {

数据对象：

n个元素构成的全集X

数据关系：

$\{<i,j>|i,j\in X\}$表示i、j属于同一个集合。

基本操作:

InitSet(*set*, *n*)：

建立n个不相交的集合$X_1,\ldots,X_n$，其中每个集合初始只有一个元素：$X_i=\{i\},\forall i=1,\ldots,n$。

DestorySet(*set*)：

释放不相交集*set*所占用的所有空间。

Find(*set*, *x*)：查询元素*x*所在的集合。

Union(set, *x*, *y*)：合并元素*x*和元素*y*所在的集合。

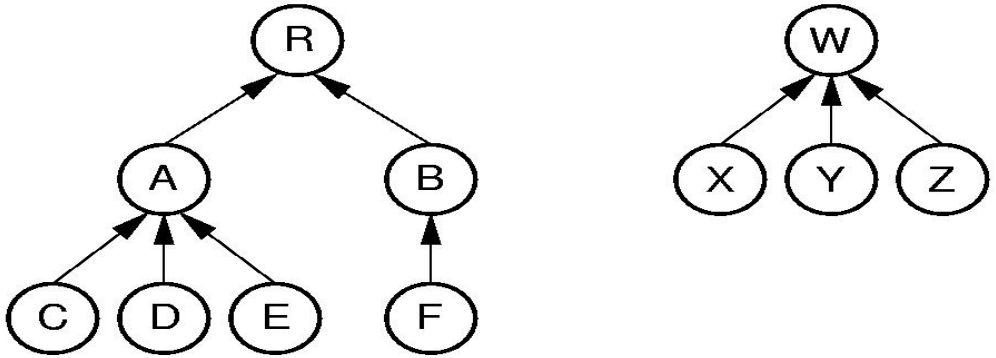
}

9.3 不相交集的存储实现

- **思考：Find操作查询元素所在的集合，输入是一个元素，输出是一个集合**
  - **如何表示一个集合?**
  - **规定每个集合中X需要有一个“代表”元素$a \in X$，该元素的编号即为集合编号，即：$\mathbf{Find}(x) = a, \forall x \in X$**
  - **例：**
    - **对于划分{{1, 2, 3, 4, 5}, {6, 7}, {8}}，可规定集合{1, 2, 3, 4, 5}代表元素为1，则Find(2)=Find(3)=Find(4)=Find(5)=1**

9.4 不相交集的基本运算实现

- **不相交集可利用父节点表示的森林数据结构实现**
  - **森林中的每棵树代表一个不相交集的集合，其“代表”元素是根结点**
  - **每个元素*x*只需维护其父亲结点*x.parent*。**
    - **特别地，规定根结点的父亲结点是其本身，即*x.parent=x*。**



Parent's Index		0	0	1	1	1	2		7	7	7
Label	R	A	B	C	D	E	F	W	X	Y	Z
Node Index	0	1	2	3	4	5	6	7	8	9	10

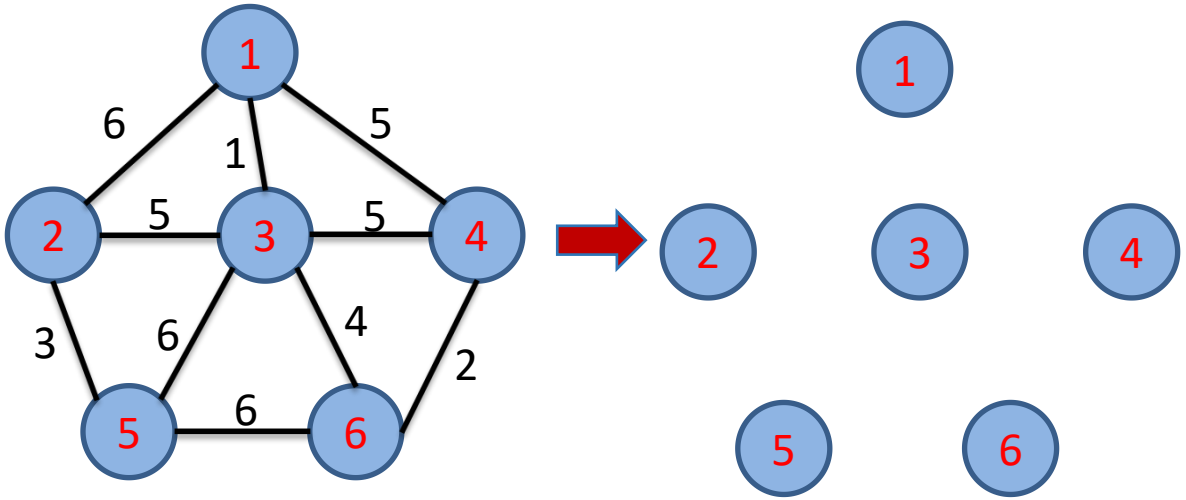
9.4 不相交集的基本运算实现

- **基本操作的实现**
  - **InitSet操作：只需定义n棵仅含根结点的树。**

算法9-1：初始化不相交集InitSet(*set*, n)

输入：元素的数量n，不相交集set={1, 2, ..., n}
输出：初始化后的集合set。

1. **for** 每个元素 $x \in set$ **do**
2. | $x.parent \leftarrow x$
3. **end**



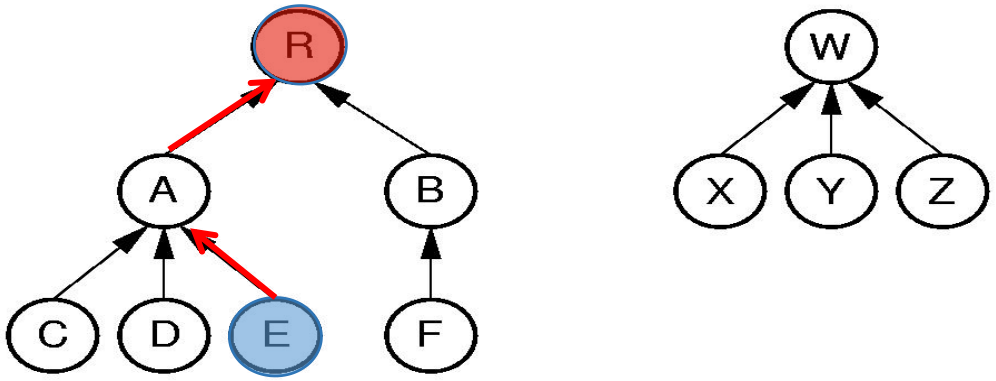
9.4 不相交集的基本运算实现

- **基本操作的实现**
  - **InitSet操作只需定义n棵仅含根结点的树。**
  - **Find操作只需找到要查询的元素*x*对应的根结点。**

算法9-2： 查找元素所在的集合Find(*set*, *x*)

输入： 不相交集*set*中待查找的元素*x*。
输出： 元素*x*所在树的根顶点。

```
while x ≠ x.parent do
    | x ← x.parent
end
return x
```



ent's Index		0	0	1	1	1	2		7	7	7
Label	R	A	B	C	D	E	F	W	X	Y	Z
Node Index	0	1	2	3	4	5	6	7	8	9	10

9.4 不相交集的基本运算实现

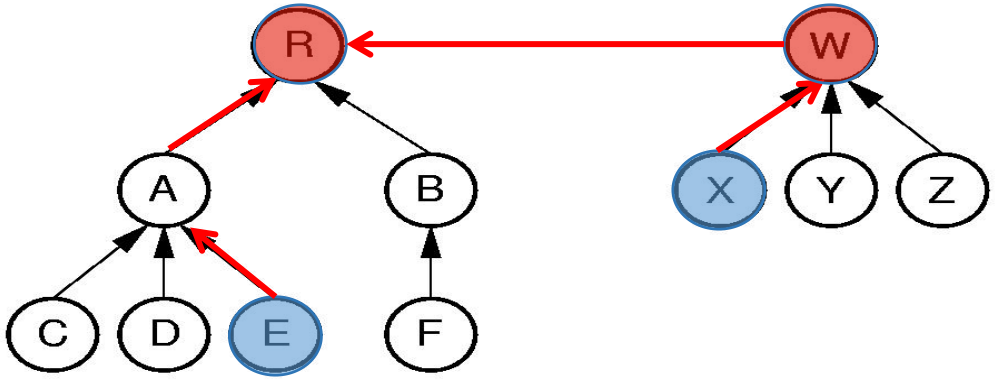
- **基本操作的实现**
  - **InitSet操作：只需定义n棵仅含根结点的树。**
  - **Find操作：只需找到要查询的元素*x*对应的根结点。**
  - **Union操作：只需将x和y元素各自对应的根结点合并**

合并E和X

算法9-3： 合并两个元素所在的集合Union(*set*, x, *y*)

输入： 不相交集*set*中的两个元素*x*和*y*。
输出： 合并*x*和*y*各自所在集合后的不相交集*set*。

1. $i \leftarrow \mathrm{Find}(set, x)$
2. $j \leftarrow \mathrm{Find}(set, y)$
3. $i.parent \leftarrow j$



dex		0	0	1	1	1	2	0	7	7	7
abel	R	A	B	C	D	E	F	W	X	Y	Z
ndex	0	1	2	3	4	5	6	7	8	9	10

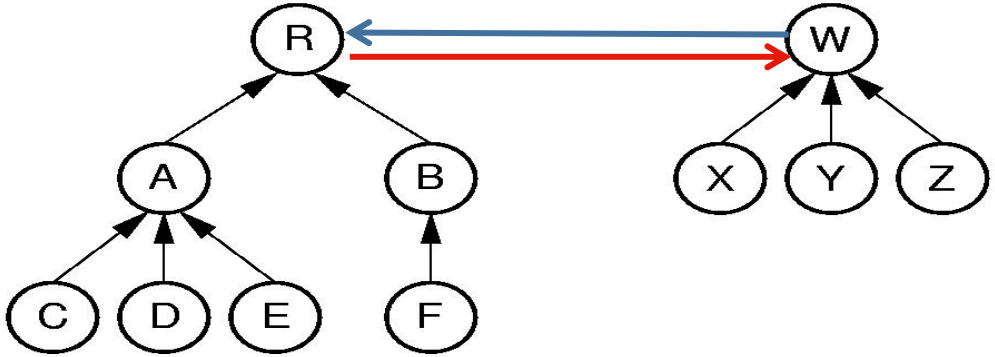
合并（Union）

- **把W合并到R的孩子结点 VS 把R合并到W的孩子结点**
  - **把R合并到W的孩子结点，会增加树的高度**
  - **从而增加Find的比较次数**

按秩合并策略：

将树的高度小的子树

合并到高度大的子树

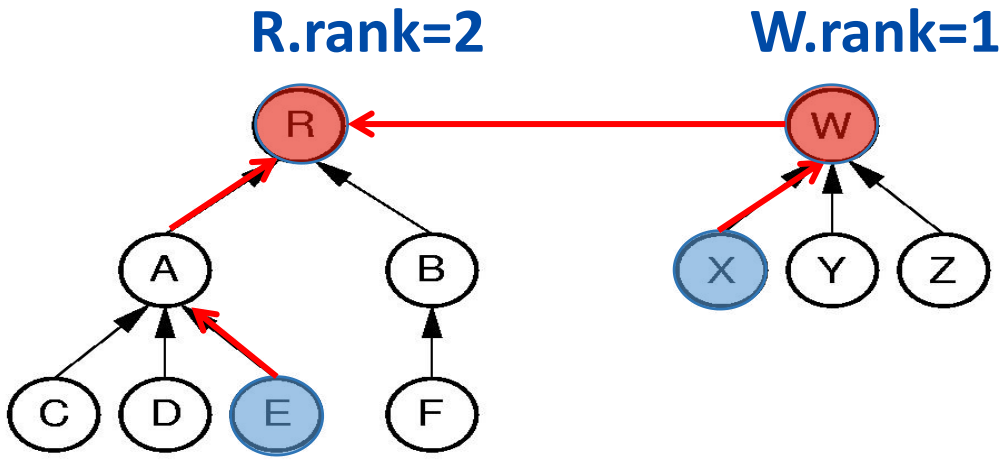


Parent's Index		0	0	1	1	1	2		7	7	7
Label	R	A	B	C	D	E	F	W	X	Y	Z
Node Index	0	1	2	3	4	5	6	7	8	9	10

9.4.1 按秩合并

- **秩（Rank）：树的高度-1**
  - **结点x的秩为：该结点的高度-1**
  - **单个结点构成的树的根结点秩为0**
- **按秩合并策略**
  - **每个结点引入一个秩：x.rank**
  - **总是把秩小的树根合并到秩大的树根**
  - **合并前的秩不同，合并后的秩不变**
  - **合并前的秩相同，合并后的秩+1**

合并后，R和W的秩有没有发生改变?



	0	0	1	1	1	2		7	7	7
R	A	B	C	D	E	F	W	X	Y	Z
0	1	2	3	4	5	6	7	8	9	10

9.4.1 按秩合并

- **按秩合并**
  - **合并前的秩不同，合并后的秩不变**
  - **合并前的秩相同，合并后的秩+1**

算法9-4： 初始化采用按秩合并策略的不相交集InitSet(*set*, n)

输入： **元素的数量**n，不相交集set={1, 2, ..., n}。
输出： 初始化后的集合set。

1. for 每个元素 $x \in set$ **do**
2. | $x.parent \leftarrow x$
3. | $x.rank \leftarrow 0$
4. End

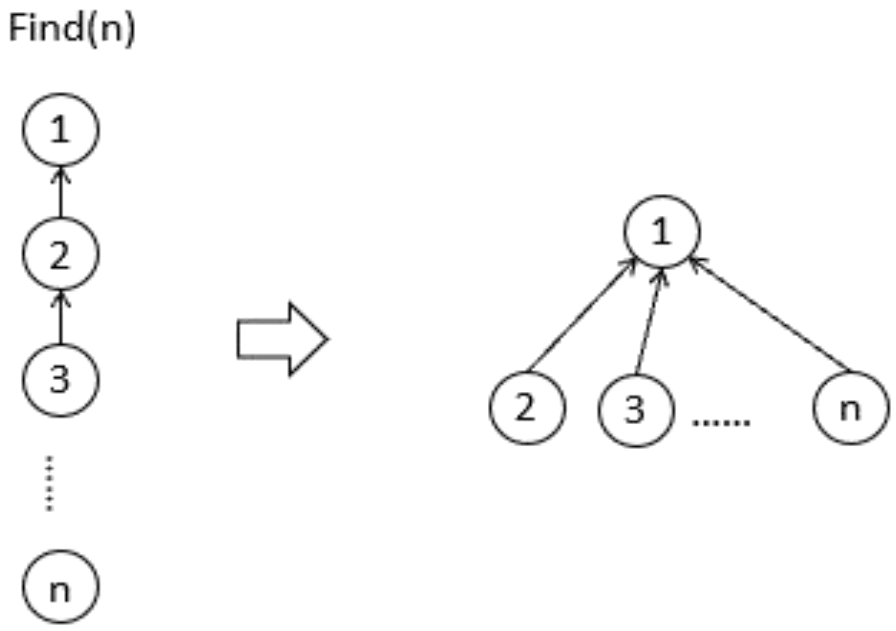
算法9-5： 利用按秩合并策略合并两个元素所在的集合Union(*set*, x, *y*)

输入： 不相交集set中的两个元素x和y。
输出： 合并x和y各自所在集合后的不相交集set。

1. $i \leftarrow \text{Find}(set, x)$
2. $j \leftarrow \text{Find}(set, y)$
3. if $i.rank > j.rank$ **then**
4. | $j.parent \leftarrow i$
5. else if $i.rank < j.rank$ **then**
6. | $i.parent \leftarrow j$
7. else $//i.rank = j.rank$
8. | $i.parent \leftarrow j$
9. | $j.rank \leftarrow j.rank + 1$
10.end

9.4.2 路径压缩

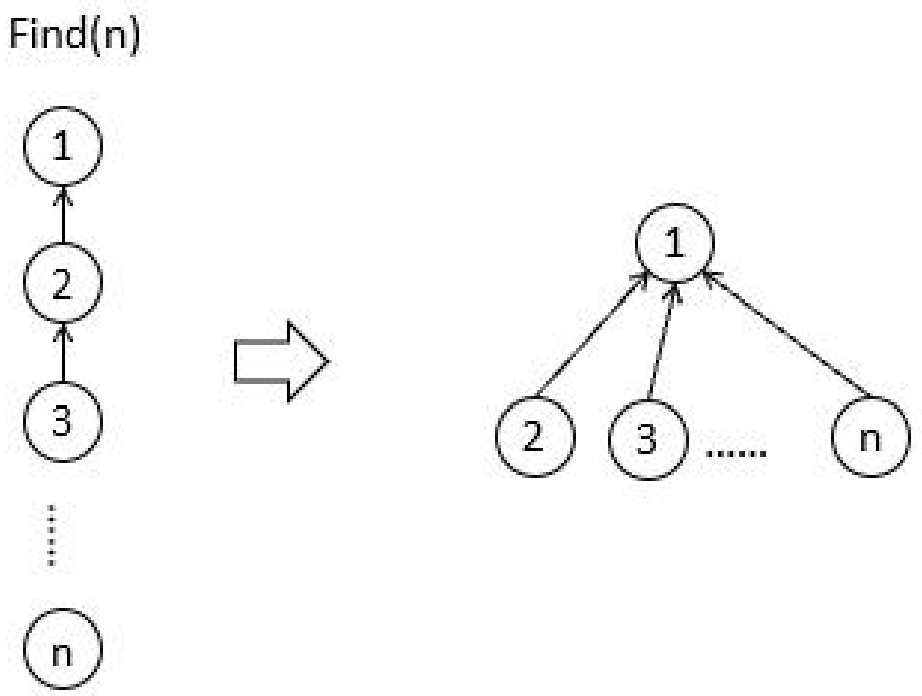
- **路径压缩（Path Compression）**
  - **假设某次查找时，路径深度过大，导致该次查找的时间成本很高。**
  - **调整森林的结构，将查找路径上的所有结点直接连接至根。**
  - **实现算法：**
    - **查找操作时，递归地查找到根r，之后逐层返回，并将路径上所有结点的父节点设为r**



路径压缩

路径压缩

- **基于路径压缩的查找**



路径压缩

算法9-6: 以路径压缩策略查找元素所在的集合 Find(*set*, *x*)

输入： 不相交集*set*中待查找的元素*x*。
输出： 元素*x*所在树的根结点。

1. **if** $x \neq x.parent$ **then**
2. | $x.parent \leftarrow$ Find(*set*, *x.parent*)
3. **end**
4. **return** *x.parent*

按秩合并与路径压缩

• 示例：同时采用按秩合并和路径压缩策略

初始时，所有结点均不相连，结点的秩均为0

Union(5, 1)时，两结点具有相同的秩，此时将5合并到1，并将结点的秩置为1

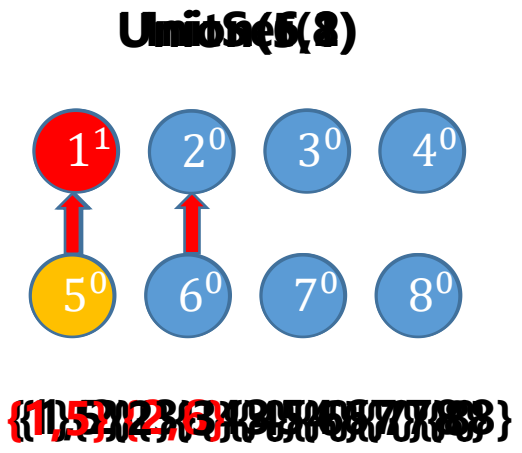
Union(6, 5)时，两棵秩为1的树合并成为一棵秩为2的树

Union(3, 6)时，首先执行的Find(6)进行了路径压缩，将6直接连接至1

*此时2成为叶子结点，但其秩未发生改变。结点1的秩也仍是2

合并操作将两棵秩为2的树合并成一棵秩为3的树

树有2^3=8个结点，在可能出现的秩为3的树中是结点数目最少的



采用了按秩合并和路径压缩的不相交集运行示例

(*图中i^r表示编号为i，秩为r的顶点)

按秩合并与路径压缩

• 示例：同时采用按秩合并和路径压缩策略

初始时，所有结点均不相连，结点的秩均为0

Union(5, 1)时，两结点具有相同的秩，此时将5合并到1，并将结点的秩置为1

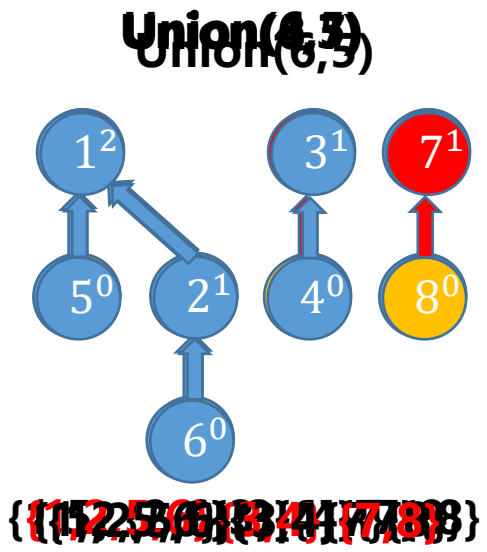
Union(6, 5)时，两棵秩为1的树合并成为一棵秩为2的树

Union(3, 6)时，首先执行的Find(6)进行了路径压缩，将6直接连接至1

*此时2成为叶子结点，但其秩未发生改变。结点1的秩也仍是2

合并操作将两棵秩为2的树合并成一棵秩为3的树

树有2^3=8个结点，在可能出现的秩为3的树中是结点数目最少的



采用了按秩合并和路径压缩的不相交集运行示例

(*图中i^r表示编号为i，秩为r的顶点)

按秩合并与路径压缩

• 示例：同时采用按秩合并和路径压缩策略

初始时，所有结点均不相连，结点的秩均为0

Union(5, 1)时，两结点具有相同的秩，此时将5合并到1，并将结点的秩置为1

Union(6, 5)时，两棵秩为1的树合并成为一棵秩为2的树

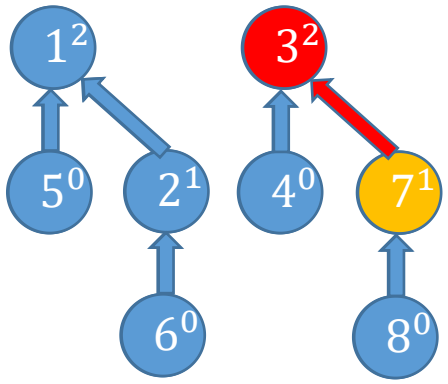
Union(3, 6)时，首先执行的Find(6)进行了路径压缩，将6直接连接至1

*此时2成为叶子结点，但其秩未发生改变。结点1的秩也仍是2

合并操作将两棵秩为2的树合并成一棵秩为3的树

树有2^3=8个结点，在可能出现的秩为3的树中是结点数目最少的

Union(7,3)



{1,2,5,6}{3,4,7,8}

采用了按秩合并和路径压缩的不相交集运行示例

(*图中i^r表示编号为i，秩为r的顶点)

按秩合并与路径压缩

- **示例：同时采用按秩合并和路径压缩策略**

初始时，所有结点均不相连，结点的秩均为0

Union(5, 1)时，两结点具有相同的秩，此时将5合并到1，并将结点的秩置为1

Union(6, 5)时，两棵秩为1的树合并成为一棵秩为2的树

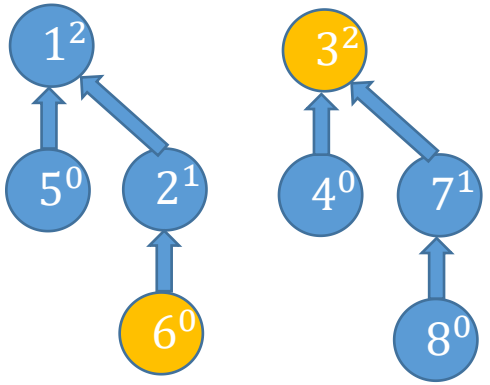
Union(3, 6)时，首先执行的Find(6)进行了路径压缩，将6直接连接至1

***此时2成为叶子结点，但其秩未发生改变。结点1的秩也仍是2**

合并操作将两棵秩为2的树合并成一棵秩为3的树

树有2^3=8个结点，在可能出现的秩为3的树中是结点数目最少的

Union(3,6)



{1,2,5,6}{3,4,7,8}

采用了按秩合并和路径压缩的不相交集运行示例

(*图中i^r表示编号为i，秩为r的顶点)

按秩合并与路径压缩

- **示例：同时采用按秩合并和路径压缩策略**

初始时，所有结点均不相连，结点的秩均为0

Union(5, 1)时，两结点具有相同的秩，此时将5合并到1，并将结点的秩置为1

Union(6, 5)时，两棵秩为1的树合并成为一棵秩为2的树

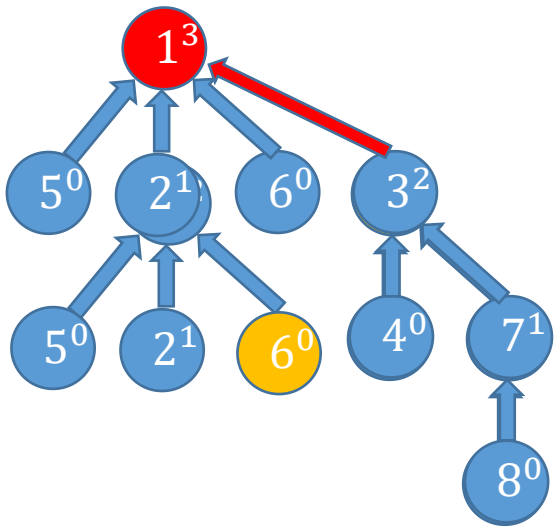
Union(3, 6)时，首先执行的Find(6)进行了路径压缩，将6直接连接至1

合并操作将两棵秩为2的树合并成一棵秩为3的树

树有2^3=8个结点，在可能出现的秩为3的树中是结点数目最少的

***此时2成为叶子结点，但其秩未发生改变。结点1的秩也仍是2**

Union(3,6)



{1,2,5,3,6,4,5,4,7,8}

采用了按秩合并和路径压缩的不相交集运行示例

(*图中i^r表示编号为i，秩为r的顶点)

不相交集算法

```
class ParPtrTree {
private:
   int *array;                          // node array
   int size;                            // size of node array
   int *rank;                           // height of tree - 1
public:
   ParPtrTree(int);
   ~ParPtrTree()   {  delete []array;   delete []rank; }
   int FIND(int) const;                 // find root
   void UNION(int, int );               // merge equivalence
   bool differ(int, int);               // true if not in same tree
}
int ParPtrTree::FIND(int curr) const {
  if (array[curr] == curr)              // root node
     return curr;
  return array[curr] = FIND(array[curr]);      // path compression
}
```

不相交集算法

```
// Return TRUE if nodes in different trees
bool ParPtrTree::differ(int a, int b)
{
  int root1 = FIND(a);        // Find root for a
  int root2 = FIND(b);        // Find root for b
  return root1 != root2;      // Compare roots
}
void ParPtrTree::UNION(int a, int b) {
  int root1 = FIND(a);    // Find root for a
  int root2 = FIND(b);    // Find root for b
  if (root1 == root2)     return;
  if (rank[root1] > rank[root2])          //按秩合并
       array[root2] = root1;
  else {
       array[root1] = root2;
      if (rank[root1] == rank[root2])
         rank[root2]++;
 }}
```

不相交集算法

- **不相交集算法，也称为并查集算法**
  - 主要的操作就是合并与查询
- **时间复杂度分析**
  - 实现不相交集非常简单，但分析其时间复杂度却非常困难。
  - 虽然单个操作的时间复杂度较高，最坏可达$O(log\,n)$。
  - 然而，由于路径压缩策略的存在，每次访问完一条路径后，下次访问该路径上结点的时间复杂度就会降低，因此算法整体的时间复杂度仍然较低。

不相交集算法

- **可使用均摊分析技巧，分析m个操作的总时间复杂度**
  - **可以证明,同时使用按秩合并和路径压缩后，n个元素组成的不相交集上进行m次Find/Union操作的时间复杂度为$O(m\alpha(n))$，即每次操作的均摊时间复杂度为$O(\alpha(n))$。**
    - $\alpha(n)$是增长非常缓慢的函数，对$n < 10^{80}$均有$\alpha(n) \leq 4$。
    - 10^{80}与可观测的宇宙中原子的数目具有相当的数量级
    - 因此，Find/Union操作几乎具有常数时间复杂度。
    - 尽管如此，不能称Find/Union具有$O(1)$的时间复杂度，因为
    - $n \to \infty$时, 仍有$\alpha(n) \to \infty$

9.5 不相交集应用：Kruskal算法

```
class KruskElem{
public:
  int from, to, distance;
  KruskElem() {
    from=to=distance=-1;
  }
  KruskElem(int f,int t,int d) {
    from=f; to=t; distance=d;
  }
  bool operator<(const KruskElem &elem) {
    return distance < elem.distance;
  }
};

void Kruskal(Graph* G){              //kruskal’s MST algorithm
  ParPtrtree A(G->n());              //等价类数组
  std::priority_queue<KruskElem, std::vector<KruskElem>> minHeap;    //最小堆
```

Kruskal算法实现

```
for (int i=0;i<G->n();i++)                          //Initialization
  for (int w=G->first(i);w<G->weight(i,w)){
    KruskElem temp(i, w, G->weight(i,w));
    minHeap.push(temp);
  }

int numMST=G->n();                                  //初始，numMST个等价类
for (int i=0; numMST>1; i++){
  KruskElem temp = minHeap.top();                   //取堆顶元素，即图中最小权边
  minHeap.pop();
  int v=temp.from;
  int u=temp.to;
  if(A.differ(v,u)){
     A.UNION(v,u);
     AddEdgetoMST(v,u);
     numMST--;
  }
}}
```

将所有边加入最小堆

取出权值最小的边，如果边的两个顶点不在同一棵生成树中，则把边加入生成树

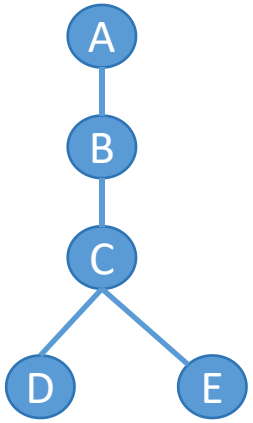
以边构建最小值堆 O(|E|)

查找权值最小的边 O(log|E|)

判断是否存在回路 O(1)

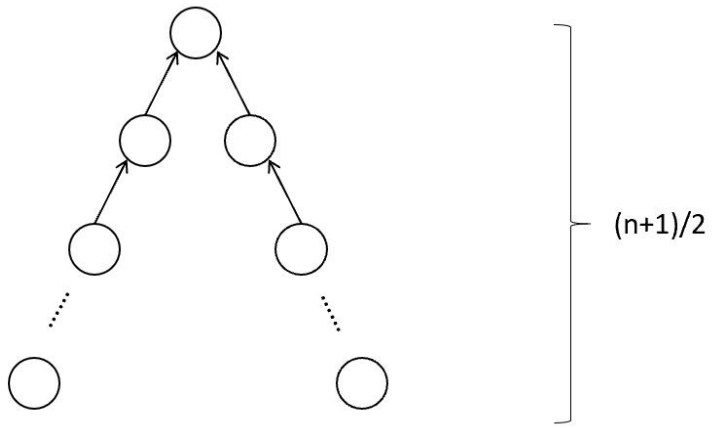
9.5 不相交集应用：最近公共祖先问题

- **公共祖先**
  - 一棵树中，所有同时是u和v祖先的结点
- **最近公共祖先（LCA，Lowest Common Ancestor）**
  - 公共祖先中**深度最大（离根最远）**的结点
  - 给定任意两个结点u和v，公共祖先构成一条以根结点为起点的链，而其中最近公共祖先是**唯一的**。
    - D和E的公共祖先包括：A，B，C
    - D和E的最近公共祖先为C



9.5 不相交集应用：最近公共祖先问题

- **LCA问题可用蛮力算法求解**
  - 首先，从结点u回溯至根，将路径上每个结点均做标记；
  - 然后，从结点v回溯至根，路径上遇到的**第一个有标记的结点**即为结点u、v的最近公共祖先
- **蛮力算法的时间复杂度较高**
  - 最坏情况下，时间复杂度可达O(n)
- **可基于不相交集算法快速求解**
  - Tarjan算法



最近公共祖先问题蛮力算法的坏情况

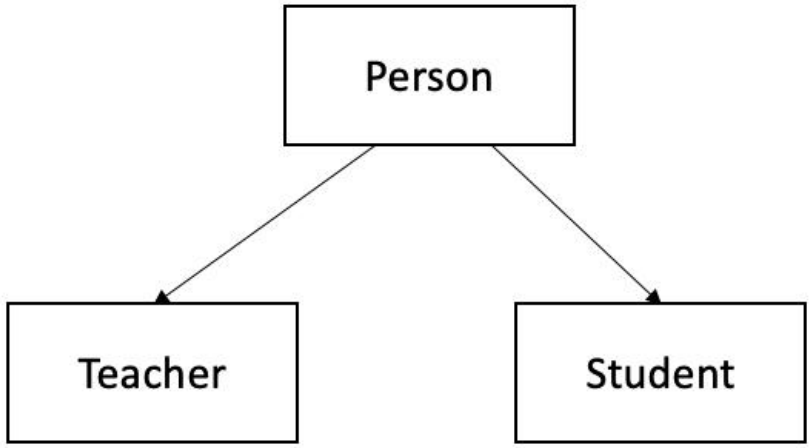
9.7 应用场景:面向对象的编程语言

- **不相交集算法可用于面向对象编程语言中多态的解析**
  - **下列C++代码定义了People、Teacher、Student三个类，其继承关系如图**

```
class Person { ... };

class Teacher : public Person { ... };

class Student : public Person { ... };
```



三个类间的继承关系

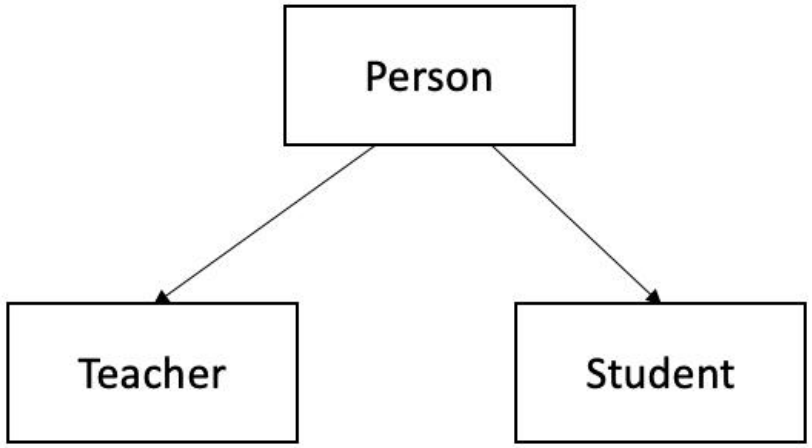
9.7 应用场景:面向对象的编程语言

- **不相交集算法可用于面向对象编程语言中多态的解析**
  - **假如定义了如下函数Talk**

```
string talk(Person a, Person b);
string talk(Teacher a, Teacher b);
string talk(Student a, Student b);
```

  - **talk(alice, bob)调用哪个接口?**

```
Teacher alice;      Student bob;
talk(alice, bob);
```



三个类间的继承关系

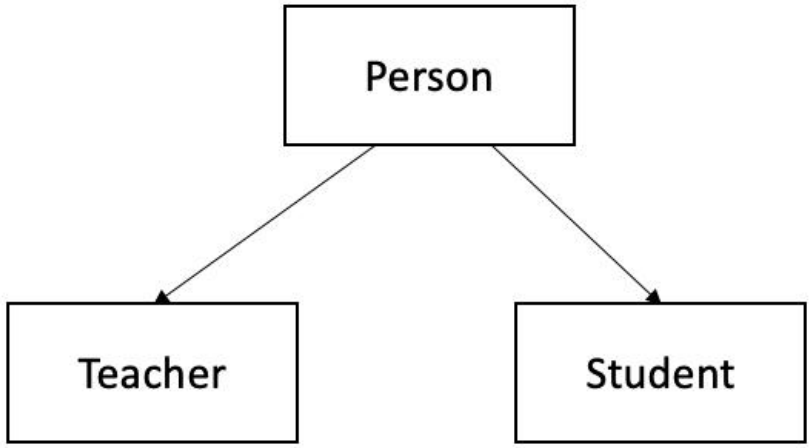
9.7 应用场景:面向对象的编程语言

- **不相交集算法可用于面向对象编程语言中多态的解析**
  - **talk(alice, bob)调用哪个接口?**

    Teacher alice; Student bob;

    talk(alice, bob);

  - **编译器会寻找两个类的最近公共祖先，来调用两个People之间的Talk函数**
  - **最近公共祖先可以用不相交集来解决**



三个类间的继承关系